



CS401 MPP

W3D2 Overview

Functional Programming in Java 8

Overview of Today

- Features and Benefits of Functional Programming
- Functional Interfaces
- Closures
- Lambda Expressions
- Parameters and Free Variables
- Naming Lambdas
- Syntax Shortcuts

Wholeness Statement

- The declarative style of functional programming makes it possible to write methods (and programs) just by declaring what is needed, without specifying the details of how to achieve the goal. Including support for functional programming in Java makes it possible to write parts of Java programs more concisely, in a more readable way, in a more threadsafe way, in a more parallelizable way, and in a more maintainable way, than ever before.
- Maharishi's Science of Consciousness: Just as a king can simply declare what he wants – a banquet, a conference, a meeting of all ministers – without having to specify the details about how to organize such events, so likewise can one who is awake to the home of all the laws of nature, the “king” among laws of nature, command those laws and thereby fulfill any intention. The royal road to success in life is to bring awareness to the home of all the laws of nature, through the process of transcending, and live life established in this field.



CS401 MPP

Functional Programming in Java 8

Features and Benefits of Functional Programming

Features of Functional Programming

- Programs are declarative (“what”) rather than imperative (“how”).
 - Makes code more self-documenting
 - the sequence of function calls mirrors precisely the requirements
- Functions have referential transparency
 - two calls to the same method are guaranteed to return the same result

```
int x;  
int mutate() {  
    ++x;  
    return x;  
}
```

Lacks referential transparency

Creates a *side effect*

Features of Functional Programming

- Functions do not cause a change of state
 - in an OO language, this means that functions do not change the state of their enclosing object (by modifying instance variables)
 - In general, functions do not have side effects; they compute what they are asked to compute and return a value, without modifying their environment (modifying the environment is a side effect). [See example, previous slide]
- Functions are first-class citizens.
 - This means in particular that it is possible to use functions in the same way objects are used in an OO language
 - They can be passed as arguments to other functions and can be the return value of a function

The Functional Style

- The Java 7 based examples on the following slides are not true functional programming examples because they rely on simpler methods that are not purely functional.
- But these examples illustrate the functional style at the top level. In Java SE 8, these techniques are supported in a truly functional way.

Imperative Version

```
public class MapImperative {
    List<String> uppercaselist;

    /* Transforms input list so that all strings are upper case
     * Populates and sorts the uppercaselist with these new values
     * Prints the new list
     */
    public void printAsUpperCase(List<String> strings) {
        uppercaselist = new ArrayList<>();
        for(String s : strings) {
            String t = s.trim();
            uppercaselist.add(t.toUpperCase());
        }
        Collections.sort(uppercaselist);
        System.out.println(uppercaselist);
    }

    public static void main(String[] args) {
        MapImperative mi = new MapImperative();
        List<String> list =
            Arrays.asList("billy", "tommy ", " rick ", "abdul",
                          "albert ", "isaiah");
        mi.printAsUpperCase(list);
    }
}
```


Functional Version

```
public class MapFunctional {
    public void printAsUpperCase(List<String> strings) {
        print(sort(mapUpperCase(mapTrim(strings))));
    }

    public String trim(String s) {
        return s == null ? null : s.trim();
    }

    public String toUpper(String s) {
        return s == null ? null : s.toUpperCase();
    }

    private void print(List<String> l) {
        System.out.println(l);
    }

    private List<String> sort(List<String> l) {
        Collections.sort(l);
        return l;
    }
}
```

```
// trims each string in the list
private List<String> mapTrim(List<String> l) {
    return map("trim", l);
}

// trims each string in the list
private List<String> mapUpperCase(List<String> l) {
    return map("toUpper", l);
}

public static void main(String[] args) {
    MapFunctional mf = new MapFunctional();
    List<String> list = Arrays.asList("billy", "tommy ",
        "rick ", "abdul", "albert ", "isaiah");
    mf.printAsUpperCase(list);
}

/* Emulation of Java 8's map method left off
 * See lesson8.lecture.functionalprogramming
 * for remaining code.
 */
```

Benefits of the Functional Style

- Programs are more compact, easier to write, and easier to read/understand
- Programs are thread-safe (due to no side-effects)
- Easier to demonstrate correctness of functional programs
- Easier to test; less likely that a test will fail tomorrow if it passed today (since there are no side effects)



CS401 MPP

Functional Programming in Java 8

Functional Interfaces

Functional Programming in Java

- Java designers – in agreement with many experts – decided it was time to incorporate the many benefits of the functional style into Java
- Java will remain an OO language, with all the benefits of OO for building and maintaining large-scale applications
- For specialized needs – for instance, to extract data from a large collection using SQL-like queries – a functional approach would be more efficient, easier to read, and less error-prone
- Another potential benefit: Significant increase in performance because of automatic support for parallelizing code.

Java 7's Approach

- Java 7 has a way to approximate “Functions as First-Class Citizens”.

```
public class Employee {  
    private String name;  
    private double salary;  
  
    public Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
    // getters and setters not on slide  
}
```

We want to sort a list
of Employee objects

To sort, we need a function
that compares Employees

```
int compare(Employee e1, Employee e2) {  
    return e1.name.compareTo(e2.name);  
}
```

Java 7's Approach

- The natural thing to try would be:

```
Collections.sort(list, compare);
```

- But Functions are not first-class in Java 7!
 - The Java solution is to wrap a function or method in an object that implements a simple **interface**
 - Whose **only method** is the method we wish to use

Java 7's Approach

- In this example, the **Comparator** interface can be used to represent the compare function.

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
}
```

- The **Comparator** interface is a declarative wrapper for the function compare
- An instance of this **Comparator** represents the "compare" function as an object

While we cannot pass the compare function
we can pass a Comparator object

Java 7's Approach

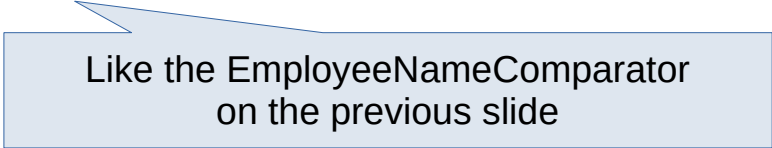
```
public class Employee {  
    private String name;  
    private double salary;  
  
    public Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
    // getters and setters not on slide  
}
```

```
public class EmployeeNameComparator  
    implements Comparator<Employee> {  
  
    @Override  
    public int compare(Employee o1, Employee o2) {  
        return o1.getName().compareTo(o2.getName());  
    }  
}
```

```
Comparator<Employee> nameCompare = new EmployeeNameComparator();  
Collections.sort(list, nameCompare);
```


Functional Interfaces and Functors

- A Comparator is called a functional interface because it has just one (abstract) method*
 - A class that implements it will contain just one function that implements an abstract method
 - any instance of it will be an object that acts like a function
- An implementation of a functional interface is called a functor



Like the EmployeeNameComparator
on the previous slide

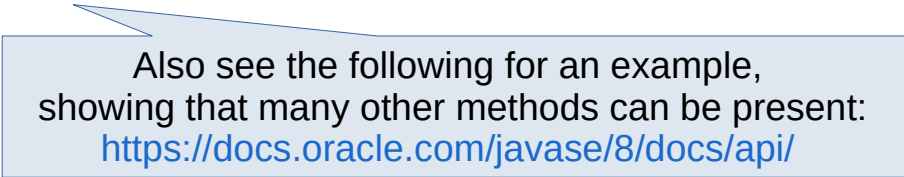
Functor

- The essential feature of **EmployeeNameComparator** is that it represents functional behavior
 - It associates to each pair (e1, e2) of Employees an integer (indicating an ordering for (e1, e2)).

$$(e1, e2) \mapsto \text{val where val} = \begin{cases} > 0 & \text{if } e1.\text{name} > e2.\text{name} \\ < 0 & \text{if } e1.\text{name} < e2.\text{name} \\ 0 & \text{if } e1.\text{name} \text{ equal to } e2.\text{name} \end{cases}$$

Note

- In reality, Comparator declares two abstract methods:
 - compare and equals
- However, equals already has an implementation in the Object class
 - The precise rule to determine whether an interface is a functional interface is: that it must have exactly one abstract method, not counting any re-declared methods from Object
 - See <https://docs.oracle.com/javase/8/docs/api/java/lang/FunctionalInterface.html>



Also see the following for an example,
showing that many other methods can be present:
<https://docs.oracle.com/javase/8/docs/api/>

Functional Interface: Consumer

- The Consumer interface, like Comparator
 - has just one abstract method (=functional interface)

```
public interface Consumer<T> {  
    void accept(T input);  
}
```

```
Consumer<String> consumer = new Consumer<String>() {  
    @Override  
    public void accept(String s) {  
        System.out.println(s);  
    }  
};  
System.out.println("-----using new forEach method-----");  
list.forEach(consumer);
```

Functional Interfaces Summary

- In Java, it is not possible to pass a function as a method argument (like compare)
- A realization of a functional interface – a functor – is a class that represents a function
 - Remember: a functional interface has only one abstract function
 - Example: EmployeeNameComparator is a functor
 - implementing the functional interface Comparator
 - Is a wrapper for the function compare



CS401 MPP

Functional Programming in Java 8

Closures

Further Refinement

- The implementation of the Comparator interface shown in the previous part has a limitation:
 - It has to use getters to retrieve values that are private
 - If the comparison of Employee objects requires information in private variables (that don't provide getters)
 - Then a Comparator is impossible to implement
 - Since it cannot access those private variables

Example

- A NameComparator and a SalaryComparator:

```
public class EmployeeNameComparator
    implements Comparator<Employee> {

    @Override
    public int compare(Employee o1, Employee o2) {
        return o1.getName().compareTo(o2.getName());
    }
}
```

```
public class EmployeeSalaryComparator
    implements Comparator<Employee> {

    @Override
    public int compare(Employee e1, Employee e2) {
        if(e1.getSalary() == e2.getSalary()) return 0;
        else if(e1.getSalary() < e2.getSalary()) return -1;
        else return 1;
    }
}
```

Both require the use of getter methods

The Need for Closures

```
public class EmployeeInfo {
    static enum SortMethod {BYNAME, BYSALARY};
    private SortMethod method;

    public EmployeeInfo(SortMethod method) {
        this.method = method;
    }
    //Comparators are unaware of the value in method
    public void sort(List<Employee> emps) {
        if(method == SortMethod.BYNAME) {
            Collections.sort(emps, new EmployeeNameComparator());
        }
        else if(method == SortMethod.BYSALARY) {
            Collections.sort(emps, new EmployeeSalaryComparator());
        }
    }
}
```

Exercise 8.1

How can we implement this sort(List<Employee> emps) method with just one Comparator object?

How can you define a Comparator so that it will be aware of the value stored in the **method** variable?

A Closure

- A closure is a block of code that behaves like a function and has access to the variables in its surrounding class
- In Java 7, instances of member, local, and anonymous inner classes are (essentially) closures, since they have full access to their enclosing object's state
- As we will see, Java 8's lambda expressions are (even better) examples of closures

A Closure

To solve the Exercise, we create a Comparator with a local inner class which acts as a closure having full access to its environment!

```
public class EmployeeInfo {
    static enum SortMethod { BYNAME, BYSALARY };
    private SortMethod method;

    public void sort(List<Employee> emps) {
        class EmployeeComparator implements Comparator<Employee> {
            @Override
            public int compare(Employee e1, Employee e2) {
                if (method == SortMethod.BYNAME) {
                    return e1.getName().compareTo(e2.getName());
                } else {
                    if(e1.getSalary()== e2.getSalary()) return 0;
                    else if(e1.getSalary() < e2.getSalary()) return -1;
                    else return 1;
                }
            }
        }
        Collections.sort(emps, new EmployeeComparator());
    }
}
```

Closure: Consumer

- The consumer object here is another closure
 - Since it's made as an anonymous inner class
 - Though in this case, the accept method does not make special use of the state of its environment

```
Consumer<String> consumer = new Consumer<String>() {  
    @Override  
    public void accept(String s) {  
        System.out.println(s);  
    }  
};  
System.out.println("-----using new forEach method-----");  
list.forEach(consumer);
```

Closures Summary

- A closure is a functor that is embedded in another class and that is aware of the state of its enclosing class
 - Example: local inner class `EmployeeComparator` is a wrapper for `compare`
 - And can use data from its enclosing class to perform comparisons



CS401 MPP

Functional Programming in Java 8

Lambda Expressions

Lambda Expressions

- Lambda Expressions are closures that may be viewed as syntax shortcuts for Java 7 closures
 - Which are inner classes of various kinds
 - Every lambda expression can be rewritten (using a lot more code) using inner classes
- Lambdas upgrade the status of (many) functions to first-class citizens

History

- Lambda notation was an invention of the mathematician A. Church in his analysis of the concept of “computable function,” long before computers had come into existence (1930s).
- Several equivalent ways of specifying a (mathematical) function:
 - $f(x, y) = 2x - y$ // this version gives the function a name (f)
 - $(x, y) \mapsto 2x - y$ // in math, this is called “maps to” notation
 - $\lambda xy. 2x - y$ // Church’s lambda notation
 - $(x, y) \rightarrow 2 * x - y$ // Java SE 8 lambda notation

In Church’s lambda notation, the function’s arguments are specified to the left of the dot, and output value to the right.

Anatomy

- A Java lambda expression (like $(x, y) \rightarrow 2 * x - y$) has three parts:
 1. parameters [zero or more in parenthesis, or one without parenthesis]
 2. $\rightarrow$ (arrow)
 3. code block [if more than one statement, enclosed in curly braces $\{ \dots \}$]

May contain free variables!
values for these are supplied by
local or instance variables

Lambda Consumer

```
Consumer<String> consumer = new Consumer<String>() {  
    @Override  
    public void accept(String s) {  
        System.out.println(s);  
    }  
};  
System.out.println("-----using new forEach method-----");  
list.forEach(consumer);
```

- Can be rewritten as:

```
System.out.println("-----using new forEach method-----");  
l.forEach(s -> System.out.println(s));
```

```
public interface Consumer<T> {  
    void accept(T input);  
}
```

The Consumer interface is implemented by:
`s -> System.out.println(s)`

Passing:

`s -> System.out.println(s)`

Is essentially the same as passing the accept method

Example: $(x,y) \rightarrow 2*x-y$

- Java SE 8 offers new functional interfaces to support many of the lambda expressions that could arise (though not all).
 - The `BiFunction<S,T,R>` interface has as its unique abstract method `apply()`
 - It returns the result of applying a function to its first two arguments (of type `S`, `T`) to produce a result (of type `R`).

```
public interface BiFunction<S, T, R> {  
    R apply(S s, T t);  
}
```

Usage

- The lambda $(x,y) \rightarrow 2 * x - y$ implements the **BiFunction** interface

```
public static void main(String[] args) {  
    BiFunction<Integer, Integer, Integer> f =  
        (x,y) -> 2 * x - y;  
    System.out.println(f.apply(2, 3)); //output: 1  
}
```

```
// Without lambdas (Java 7 style)  
public static void main(String[] args) {  
    BiFunction<Integer, Integer, Integer> f = new BiFunction<Integer, Integer, Integer>() {  
        public Integer apply(Integer x, Integer y) {  
            return 2 * x - y;  
        }  
    };  
    System.out.println(f.apply(2, 3));  
}
```

Another Lambda Example

```
public class EmployeeInfo {
    static enum SortMethod { BYNAME, BYSALARY };
    private SortMethod method;

    public void sort(List<Employee> emps) {
        class EmployeeComparator implements Comparator<Employee> {
            @Override
            public int compare(Employee e1, Employee e2) {
                if (method == SortMethod.BYNAME) {
                    return e1.getName().compareTo(e2.getName());
                } else {
                    if(e1.getSalary()== e2.getSalary()) return 0;
                    else if(e1.getSalary() < e2.getSalary()) return -1;
                    else return 1;
                }
            }
        }
        Collections.sort(emps, new EmployeeComparator());
    }
}
```

```
public class EmployeeInfo {
    static enum SortMethod { BYNAME, BYSALARY };
    private SortMethod method;

    public void sort(List<Employee> emps) {
        Collections.sort(emps, (e1, e2) -> {
            if (method == SortMethod.BYNAME) {
                return e1.getName().compareTo(e2.getName());
            } else {
                if(e1.getSalary()== e2.getSalary()) return 0;
                else if(e1.getSalary() < e2.getSalary()) return -1;
                else return 1;
            }
        });
    }
}
```

A Sample Application Of Lambdas

- Task: Extract from a list of names (Strings) a sublist containing those names that begin with a specified character, and transform all letters in such names to uppercase.

```
public List<String> findStartsWithLetterToUpper(List<String> list, char c) {  
    List<String> startsWithLetter = new ArrayList<String>();  
    for (String name : list) {  
        if (name.startsWith(" " + c)) {  
            startsWithLetter.add(name.toUpperCase());  
        }  
    }  
    return startsWithLetter;  
}
```

Imperative Style (Java 7)

A Sample Application of Lambdas

Using Lambdas and Streams

Streams will be discussed fully in lesson 9

```
public List<String> findStartsWithLetterToUpper(List<String> list, String letter) {  
    return list.stream(). // convert list to stream  
        filter(name -> name.startsWith(letter)) // returns filtered stream  
        .map(name -> name.toUpperCase()) // maps each string to a uppercase version  
        .collect(Collectors.toList()); // organizes stream into a list  
}
```

```
public List<String> findStartsWithLetterToUpper(List<String> list, String letter) {  
    return list.parallelStream(). // convert list to stream  
        filter(name -> name.startsWith(letter)) // returns filtered stream  
        .map(name -> name.toUpperCase()) // maps each string to a uppercase version  
        .collect(Collectors.toList()); // organizes stream into a list  
}
```

Parallel Processing



CS401 MPP

Functional Programming in Java 8

Parameters and Free Variables

Parameters and Free Variables

- Parameters are the arguments that are passed as arguments in the lambda expression:
 - In $(x, y) \rightarrow x * y$, the variables x, y are parameters
- Free variables are variables in the lambda expression that are not parameters and not defined inside the block of code on the right hand side of the lambda expression

```
Collections.sort(emps, (e1, e2) -> {  
    if (method == SortMethod.BYNAME) {  
        return e1.getName().compareTo(e2.getName());  
    } else {  
        if(e1.getSalary()== e2.getSalary()) return 0;  
        else if(e1.getSalary() < e2.getSalary()) return -1;  
        else return 1;  
    }  
});
```

Parameters: e1, e2
Free variables: method, SortMethod

Parameters and Free Variables

- In order for a lambda expression to be evaluated
 - Values for the free variables need to be supplied
 - Either as arguments in the method where the lambda appears
 - Or in the enclosing class
- Values that are supplied to the free variables are said to be **captured** by the lambda expression

Example

```
btn.setOnAction(evt -> System.out.println("Hello " + (username != null ? username : "World"))) );
```

- Parameter: evt
- Free Variables:
 - out (in the system class, outside the expression)
 - username (also defined outside the expression)

Create Your Own Functional Interface

```
@FunctionalInterface
public interface TriFunction<S,T,U,R> {
    R apply(S s, T t, U u);
}
```

The `@FunctionalInterface` annotation is checked by the compiler

if the interface does not contain exactly one abstract method, there is a compiler error.

```
public class MyOwn {
    public static void main(String[] args) {
        TriFunction<Integer, Integer, Integer, Integer> f =
            (x,y, z) -> x + y + z;
        System.out.println(f.apply(2, 3, 4));
    }
}
```

Not required, but strongly recommended, to enable compile-time checking.

See: [lesson8.lecture.lambdaexamples.trifunction](#)

Exercise 8.2

1. Is this a functional interface? Explain.

```
@FunctionalInterface
public interface Example1 {
    String toString();
}
```

2. Does the following code compile? Explain.

```
@FunctionalInterface
public interface Example2 {
    String toString();
    void act();
}
```

3. Define your own functional interface that could be used as a wrapper for a function that takes zero arguments and returns an integer.

- Then use it to define an object that produces random numbers
- Test your code with the given main

```
@FunctionalInterface
public interface MyIface {
    ...
}
```

```
public class MyRandGen implements MyIface {
    ...
    public static void main(String[] args) {
        MyIface mrg = new MyRandGen();
        System.out.println(mrg.produce());
    }
}
```

Exercise 8.2

1. Is this a functional interface? Explain.

```
@FunctionalInterface
public interface Example1 {
    String toString();
}
```

No – we don't count methods that override methods from Object

2. Does the following code compile? Explain.

```
@FunctionalInterface
public interface Example2 {
    String toString();
    void act();
}
```

Yes, there is just one abstract method not counting methods from Object

3. Define your own functional interface that could be used as a wrapper for a function that takes zero arguments and returns an integer.

- Then use it to define an object that produces random numbers
- Test your code with the given main

```
@FunctionalInterface
public interface MyIface {
    int produce();
}
```

```
public class MyRandGen implements MyIface {
    @Override
    public int produce() {
        Random r = new Random();
        return r.nextInt();
    }
}
```

Main Point 1

- Before Java SE 8, functions were not first-class citizens
 - which made the functional style difficult to implement.
- Prior to Java SE 8, Java approximated a function with a functional interface
 - when implemented as an inner class, objects of this type were close approximations to functions.
- In Java SE 8, these inner class approximations can be replaced by lambda expressions
 - which capture their essential functional nature: Arguments mapped to outputs.
 - With lambda expressions, it is now possible to reap many of the benefits of the functional style while maintaining the OO essence of the Java language as a whole.
- The “purification” process that made it possible to transform “noisy” one-method inner classes into simple functional expressions (lambdas) is like the purification process that permits a noisy nervous system to have a chance to operate smoothly and at a higher level.
 - This is one of the powerful benefits of the transcending process.



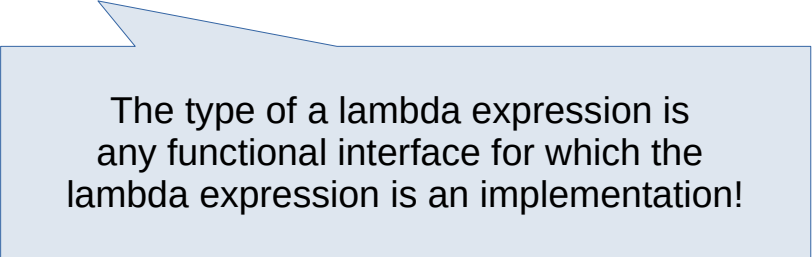
CS401 MPP

Functional Programming in Java 8

Naming Lambda Expressions

Naming Lambda Expressions

- We want to be able to reuse lambda expressions rather than rewriting the entire expression each time
 - To do so, we need to give it a name and a type
- Every object in Java has a type
 - The same is true of lambda expressions



The type of a lambda expression is any functional interface for which the lambda expression is an implementation!

Example

- Consider this Lambda Expression:

```
(Employee e1, Employee e2) -> e1.getName().compareTo(e2.getName());
```

- What functional interface has been implemented here?
- Analysis:
 - The lambda accepts two parameters – e1 and e2 –
 - Has a return value of type Integer

We must find (or create) a functional interface whose encapsulated function has these characteristics.

How to Search

- The new functional interfaces in Java 8 are contained in the package **java.util.function**
 - We can look at the JDK API docs at:
<https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html>
 - To locate a matching interface.

Java.util.function

Our first candidate:
`BiFunction<T,U,R>`

Interface Summary

Interface	Description
<code>BiConsumer<T,U></code>	Represents an operation that accepts two input arguments and returns no result.
<code>BiFunction<T,U,R></code>	Represents a function that accepts two arguments and produces a result.
<code>BinaryOperator<T></code>	Represents an operation upon two operands of the same type, producing a result of the same type as the operands.
<code>BiPredicate<T,U></code>	Represents a predicate (boolean-valued function) of two arguments.
<code>BooleanSupplier</code>	Represents a supplier of boolean-valued results.
<code>Consumer<T></code>	Represents an operation that accepts a single input argument and returns no result.

```
BiFunction<Employee, Employee, Integer> lambda  
    = (Employee e1, Employee e2) -> e1.getName().compareTo(e2.getName());
```

This works, you can type it like this

Another Possibility

- Recall that a Comparator accepts two arguments and returns an integer
 - Positive integer means “greater than”
 - Negative integer means “less than”

```
Comparator<Employee> lambda  
    = (Employee e1, Employee e2) -> e1.getName().compareTo(e2.getName());
```

This second candidate is also correct!

The Lambda can be typed either way

Question

- Which of the two should we use?
- Answer: it depends on your intended use
 - In this case it's likely the lambda was designed as a comparator for sorting
 - Only the comparator type can be used for this purpose

```
Comparator<Employee> lambda1  
    = (Employee e1, Employee e2) -> e1.getName().compareTo(e2.getName());  
BiFunction<Employee, Employee, Integer> lambda2  
    = (Employee e1, Employee e2) -> e1.getName().compareTo(e2.getName());
```

```
Collections.sort(list, lambda1);  
Collections.sort(list, lambda2);
```

This second line gives a compiler type error!

Note

- Although every lambda is a realization of a functional interface
 - the way in which the Java compiler translates a lambda into a realization of such an interface is not obvious.
 - Historically, the possibility of simply translating the lambda into an anonymous inner class was considered by the Java engineers, but was rejected for a number of reasons.
 - One reason is performance – inner classes have to be loaded separately by the class loader
 - Another is that tying lambdas to such an implementation would limit the possibility for evolution of new features of lambdas in future releases.
- You can verify that lambdas and anonymous inner classes are fundamentally different (even though very similar) by considering how the implicit object reference ``this'` is interpreted by each type:
 - In an anonymous inner class, ``this'` refers to the inner class;
 - In a lambda, ``this'` refers to the surrounding class.
 - See: <http://www.infoq.com/articles/Java-8-Lambdas-A-Peek-Under-the-Hood>



CS401 MPP

Functional Programming in Java 8

Syntax Shortcuts

Method Expressions 1 / 3

Lesson 9 will also discuss a fourth type

- **object::instanceMethod.**
- Given an object `obj` and an instance method `meth()` in `obj`, the lambda expression:

```
x -> obj.meth(x)
```

Note: `obj` is a free variable!

Can be written as:

```
obj::meth
```

- Example:

```
button.setOnAction(evt -> p.print(evt));
```



```
button.setOnAction(p::print);
```

See: `lesson8.lecture.methodreferences.objinstance.print`

Method Expressions 2 / 3

- **Class::staticMethod**
- Given a class **ClassName** and one of its static methods **meth()**, the lambda expression:

```
x -> ClassName.meth(x)
```

Can be written as:

```
ClassName::meth
```

- Example:

```
BiFunction<Integer, Integer, Double> f  
= (x,y) -> Math.pow(x, y);
```



```
BiFunction<Integer, Integer, Double> f = Math::pow;
```

See: [lesson8.lecture.methodreferences.classmethod.math](#)

Method Expressions 3 / 3

- **Class::instanceMethod**
- Given a class **ClassName** and one of its instance methods **meth()**, the lambda expression:

```
(x,y) -> x.meth(y)
```

Can be written as:

```
ClassName::meth
```

- Example:

```
(str1, str2) -> str1.compareToIgnoreCase(str2)
```



```
String::compareToIgnoreCase
```

Syntax Shortcuts via Target Typing

1. If parameter types can be inferred, they can be omitted

```
public void sort(List<Employee> emps, SortMethod method) {  
    Collections.sort(emps, (e1,e2) ->  
    {  
        if(method == SortMethod.BYNAME) {  
            return e1.name.compareTo(e2.name);  
        } else {  
            if(e1.salary == e2.salary) return 0;  
            else if(e1.salary < e2.salary) return -1;  
            else return 1;  
        }  
    });  
}
```

Because **emps** is known to be a list of **Employee**
The compiler can infer that **e1** and **e2** must
be employees that are going to be compared

See: [lesson8.lecture.lambdaexamples.comparator3](#)

Syntax Shortcuts via Target Typing

2. If a lambda has a single parameter with an inferred type
- the parentheses around the parameter can be omitted

```
Consumer consumer = str -> {    //instead of (str)
    System.out.println(str);
};
```

```
EventHandler<ActionEvent> handler = evt -> {
    System.out.println("Hello World");
};
```

Exercise 8.3

- Lambda and Method Reference Exercises
 - See PDF in lesson8.exercise_3 in InClassExercises
- Assign each lambda expressions to a variable of the appropriate type and then express as a method reference.
 - Indicate which type of method reference you are using.

Example: `(String x) -> x.toUpperCase()`

```
Function<String, String> toUpper1 = (String x) -> x.toUpperCase();  
Function<String, String> toUpper2 = String::toUpperCase;  
Method reference type: Class::instanceMethod
```



CS401 MPP

Functional Programming in Java 8

Closing Remarks

Connecting the Parts of Knowledge With the Wholeness of Knowledge

1. In Java SE 7, the only first-class citizens are objects, created from classes. The valuable techniques of functional programming and a declarative style can be approximated using functional interfaces.
 2. In Java SE 8, functions – in the form of lambda expressions – have become first-class citizens, and can be passed as arguments and occur as return values. In this new version, the advantage of functional programming with its declarative style is now supported in the language
-
3. **Transcendental Consciousness:** TC, which can be experienced in the stillness of one's awareness through transcending, is where the laws of nature begin to operate – it is the home of all the laws of nature
 4. **Impulses Within the Transcendental Field:** As TC becomes more familiar, more and more, intentions and desires reach fulfillment effortlessly, because of the hidden support of the laws of nature.
 5. **Wholeness moving within Itself:** In Unity Consciousness, one finally recognizes the universe in oneself – that all of life is simply the impulse of one's own consciousness. In that state, one effortlessly commands the laws of nature for all good in the universe.

